

Guía n.º 1.4.3

Actividad: Colecciones y manejo de excepciones en Java

Organización de datos, validación de operaciones y control de errores en programación orientada a objetos

Guía n.º	1.4.3
Nombre de la guía	Actividad de análisis y aplicación de colecciones y manejo de excepciones en Java
Sigla	DSY1102
Asignatura	Desarrollo Orientado a Objetos
Experiencia de Aprendizaje	Reconocer el uso de colecciones para almacenar objetos y aplicar manejo de excepciones para controlar errores comunes en programas Java.
Tiempo	90 minutos
Modalidad de Trabajo	Individual o en parejas
Indicadores de Logro	Distingue distintos tipos de colecciones, utiliza operaciones básicas sobre listas y conjuntos, y aplica bloques try-catch-finally y validaciones personalizadas para controlar errores.

Propósito de la guía

Desarrollar la capacidad de organizar y manipular grupos de datos en Java mediante colecciones, comprendiendo cuándo conviene usar estructuras como ArrayList, HashSet o HashMap. Además, la guía busca fortalecer el control de errores mediante el uso de excepciones, permitiendo que los programas respondan de manera segura ante situaciones inesperadas como índices inválidos, datos nulos o conversiones incorrectas.

La guía presenta un caso base relacionado con la gestión de estudiantes inscritos en un curso. A partir de este ejemplo se explican las colecciones más utilizadas, el manejo básico de excepciones y la forma de combinar ambos conceptos en soluciones orientadas a objetos.

Instrucciones generales

- Lee con atención el caso base antes de revisar el código en Java.
- Observa qué estructura de colección se utiliza en cada ejemplo y por qué.
- Distingue cuándo se agrega, busca, elimina o recorre información dentro de una colección.
- Relaciona cada posible error con la excepción correspondiente.
- Analiza el propósito de los bloques try, catch, finally y de la palabra throw.
- Completa los ejercicios finales justificando tus decisiones de modelado y control de errores.

Formato sugerido de respuesta para cada ejercicio

1. Nombre de la clase
2. Descripción breve de la responsabilidad de la clase
3. Lista de atributos
4. Lista de métodos
5. Constructor propuesto
6. Justificación breve: ¿qué datos y acciones fueron seleccionados y por qué?



Caso base: Registro de estudiantes en un curso

Situación: Un sistema académico necesita registrar estudiantes inscritos en un curso. Se requiere almacenar sus nombres, evitar duplicados en ciertos listados, asociar cada estudiante con su correo institucional y controlar posibles errores al buscar o acceder a información inexistente.

Consigna: Analiza los siguientes ejemplos en Java y observa cómo se aplican colecciones y manejo de excepciones dentro de una misma solución.

Uso de ArrayList	<pre>import java.util.ArrayList; public class Curso { private String nombre; private ArrayList<String> estudiantes; public Curso(String nombre) { this.nombre = nombre; this.estudiantes = new ArrayList<>(); } public void agregarEstudiante(String estudiante) { estudiantes.add(estudiante); } public void mostrarEstudiantes() { for (String estudiante : estudiantes) { System.out.println(estudiante); } } }</pre>
Uso de HashSet y HashMap	<pre>import java.util.HashMap; import java.util.HashSet; HashSet<String> correosRegistrados = new HashSet<>(); HashMap<String, String> correoPorEstudiante = new HashMap<>(); correosRegistrados.add("ana@duoc.cl"); correoPorEstudiante.put("Ana", "ana@duoc.cl");</pre>
Manejo básico de excepción	<pre>public String obtenerEstudiante(int indice) { try { return estudiantes.get(indice); } catch (IndexOutOfBoundsException e) { return "Índice fuera de rango"; } }</pre>
Descripción general	La solución utiliza distintas colecciones según la necesidad: una lista para mantener estudiantes en orden, un conjunto para evitar duplicados y un mapa para asociar un nombre con un correo. Además, se incorpora manejo de excepciones para evitar que el programa falle al acceder a posiciones inexistentes.

Conceptos clave trabajados en la guía



Colección	Una colección es una estructura que permite almacenar y manipular varios elementos. En Java existen diferentes tipos según la necesidad del problema.
ArrayList	ArrayList permite guardar elementos en orden y acceder a ellos mediante índices. Es útil cuando interesa mantener secuencia y permitir repeticiones.
HashSet	HashSet almacena elementos sin orden fijo y evita duplicados. Es apropiado cuando interesa garantizar unicidad.
HashMap	HashMap almacena pares clave-valor. Es útil cuando se necesita asociar un dato con otro, por ejemplo un nombre con un correo.
Excepción	Una excepción es un evento que interrumpe el flujo normal del programa debido a un error o situación inesperada.
try-catch-finally	try contiene el código riesgoso, catch captura la excepción y finally ejecuta acciones finales independientemente del resultado.
throw	throw permite lanzar una excepción de forma manual cuando una condición no cumple las reglas del sistema.

¿Cómo se observan las colecciones en este ejemplo?

Las colecciones aparecen como herramientas para resolver necesidades distintas dentro del mismo sistema. ArrayList se utiliza para mantener una lista ordenada de estudiantes, HashSet para controlar que ciertos datos no se repitan y HashMap para asociar una clave con un valor relacionado. Esto demuestra que no todas las estructuras sirven para lo mismo y que la elección depende del problema.

¿Cómo se observa el manejo de excepciones en este ejemplo?

El manejo de excepciones se observa cuando el programa intenta acceder a un índice que podría no existir dentro de la lista. En lugar de permitir que el sistema se detenga, se utiliza un bloque try-catch para capturar el error y responder de forma controlada. Esto mejora la seguridad y la claridad del programa frente a situaciones inesperadas.

Relación entre estructuras y situaciones de uso

Elemento	Ejemplo en la guía	Función dentro del diseño
ArrayList	Lista de estudiantes	Permite almacenar elementos en orden y recorrerlos fácilmente.
HashSet	Correos registrados	Evita que un dato se repita dentro del conjunto.
HashMap	Nombre y correo	Relaciona una clave con un valor asociado.
try-catch	Acceso por índice	Controla errores cuando una operación puede fallar.
throw	Validación de datos	Permite detener una operación inválida con un mensaje claro.

Ejemplo de validación y uso



Lanzamiento manual de excepción	<pre>public void agregarNota(double nota) { if (nota < 1.0 nota > 7.0) { throw new IllegalArgumentException("La nota debe estar entre 1.0 y 7.0"); } System.out.println("Nota registrada: " + nota); }</pre>
Uso desde main	<pre>public class Main { public static void main(String[] args) { Curso curso = new Curso("Programación"); curso.agregarEstudiante("Ana"); curso.agregarEstudiante("Luis"); curso.mostrarEstudiantes(); System.out.println(curso.obtenerEstudiante(5)); } }</pre>

Explicación de la solución propuesta

Selección de colecciones	Se utilizan distintas estructuras porque cada una resuelve un tipo de problema específico: orden, unicidad o asociación entre datos.
Control de errores	Las excepciones permiten anticipar fallos comunes y responder con mensajes o acciones alternativas sin detener el programa.
Validación de reglas	El uso de throw ayuda a reforzar reglas del dominio, por ejemplo impedir registrar una nota fuera de rango.
Legibilidad del código	Separar la lógica de almacenamiento y la lógica de manejo de errores mejora la organización y comprensión del programa.

Actividades de aplicación

Ejercicio 1: Selección de colección adecuada

Situación: Un sistema debe guardar nombres de libros prestados en orden de llegada, evitar repetir códigos de préstamo y asociar cada estudiante con su número de matrícula.

Consigna: Indica qué colección usarías en cada caso y justifica por qué ArrayList, HashSet o HashMap sería la opción más adecuada.

Clase propuesta	
Atributos	
Métodos	
Constructor	



Justificación	
----------------------	--

Ejercicio 2: Propuesta de clase con ArrayList

Situación: Una biblioteca desea registrar varios títulos dentro de una categoría específica.

Consigna: Propón una clase CategoriaLibros o un nombre equivalente. Define al menos 4 atributos, 3 métodos y un constructor, incorporando una colección apropiada para guardar títulos.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	

Ejercicio 3: Uso de HashMap

Situación: Una aplicación académica necesita asociar el nombre de cada asignatura con su promedio final.

Consigna: Diseña una propuesta de clase que utilice un HashMap para almacenar esa relación. Explica qué claves y valores manejarías y qué métodos serían necesarios.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	



Ejercicio 4: Detección de errores y excepciones

Situación: Un método intenta dividir dos números ingresados por el usuario y también acceder a un elemento de una lista.

Consigna: Identifica qué excepciones podrían producirse en ese escenario y explica cómo las controlarías usando try-catch.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	

Ejercicio 5: Validación con throw

Situación: Un sistema de ventas no debe permitir registrar cantidades negativas ni precios menores o iguales a cero.

Consigna: Propón un método en Java que valide esos datos y lance una excepción cuando no cumplan las reglas del problema. Explica por qué esa validación es importante.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	



Criterios de evaluación sugeridos

Categoría	% logro	Descripción de niveles de logro
Comprensión de colecciones	25%	Reconoce diferencias entre ArrayList, HashSet y HashMap y las aplica según la necesidad del problema.
Manejo de excepciones	25%	Identifica errores posibles y propone mecanismos adecuados de control mediante excepciones.
Aplicación en clases y métodos	25%	Integra colecciones y validaciones dentro de propuestas coherentes de clases en Java.
Claridad y justificación	25%	Presenta respuestas ordenadas, comprensibles y bien fundamentadas según el contexto planteado.

Cierre

Al finalizar esta guía, el estudiante debería ser capaz de reconocer qué colección conviene utilizar según el tipo de dato y operación requerida, así como controlar errores frecuentes mediante excepciones para construir programas en Java más seguros, organizados y fáciles de mantener.